

Implementação de Processos, Threads e Comunicação Básica (Sockets) - Etapa 2

Nesta etapa do projeto, nós nos concentramos em dar vida às entidades do meu "Painel de Monitoramento Climático". Fizemos isso usando **threads** para coleta de dados e estabelecendo **comunicação TCP via sockets** com um servidor central.

Minhas Entidades como Threads: Por que Escolhi?

Eu escolhi usar **threads** para representar os coletores de dados climáticos por estado. Minha decisão se baseou em alguns pontos importantes:

- **Compartilhamento de Memória:** Threads operam no mesmo espaço de memória do processo principal. Isso me facilitou muito o acesso ao DataFrame de cidades, que já estava carregado.
- **Custo/Performance:** Threads são mais leves e rápidas de criar do que processos. Como eu tinha que lidar com um número de estados relativamente limitado, o custo de criar threads era baixo. Elas também são eficientes para tarefas que esperam por Entrada/Saída (I/O-bound), como chamadas de API e comunicação via socket.
- **Simplicidade:** Para este cenário, onde cada thread gerencia sua própria conexão e dados, achei que o modelo de threads oferecia uma implementação mais direta, sem a complexidade adicional da comunicação entre processos.

Minha Comunicação TCP via Sockets

Eu implementei a comunicação entre o **cliente de coleta de dados** (onde minhas threads rodam) e um **servidor central de clima**. Para isso, utilizei **sockets TCP** para garantir a entrega confiável dos dados.

Por que TCP?

Dados climáticos como temperatura, umidade e a lista de cidades são informações cruciais. Elas exigem **confiabilidade**, ou seja, a garantia de que os dados cheguem completos e na ordem correta. O TCP me oferece essa garantia, ao contrário do UDP, que prioriza a velocidade mas pode perder pacotes, o que não seria aceitável aqui.

Como Estructurei Minhas Mensagens:

Eu formatei minhas mensagens em **JSON** e as enviei como strings, usando um terminador de linha (`\n`).

- **No Cliente:** Primeiro, converto os dados para JSON, adiciono o `\n`, codifico tudo para bytes e depois envio.
- **No Servidor:** Recebo os bytes, os acumulo até encontrar o `\n`, decodifico para uma string, faço o parse do JSON e então envio uma confirmação de volta.

Como Demonstrei a Interação Básica

Meu sistema funciona da seguinte maneira:

1. O **Servidor** inicia e fica aguardando por conexões.
2. O **Cliente** começa coletando a lista de cidades do IBGE.
3. Com essa lista, eu crio uma **thread** para cada estado.
4. Cada thread, então:
 - Simula a coleta de dados climáticos para o estado que lhe foi atribuído.
 - Conecta-se ao servidor central via TCP.
 - Envia os dados climáticos no formato JSON.
 - Aguarda e recebe uma confirmação do servidor.
5. O **Servidor** processa os dados que recebe de cada thread.

Essa interação básica me permitiu estabelecer a base para um sistema distribuído. Nela, as entidades (minhas threads clientes) podem reportar informações de estado para um componente central (o servidor). Eu também adicionei uma lógica de re-tentativa no cliente para torná-lo mais robusto a falhas temporárias de conexão.

Como Rodar o Meu Projeto

Para testar o projeto, siga estes passos:

Pré-requisitos:

Certifique-se de ter o Python 3.x instalado. Você precisará das bibliotecas `requests` e `pandas`. Eu as instalei usando pip:

Bash

```
pip install requests pandas
```

Estrutura de Arquivos:

Eu organizei o código em dois arquivos na mesma pasta:

1. **servidor_clima.py**: Este é o código do servidor TCP.
2. **cliente_clima.py**: Este é o código do cliente com as threads que fazem a coleta e o envio.

Passos para Execução:

Iniciar o Servidor: Abra um terminal (ou prompt de comando). Vá até o diretório onde você salvou os arquivos. Execute o script do servidor:

Bash

```
python servidor_clima.py
```

1. O terminal do servidor mostrará a mensagem **--- Servidor TCP de Clima escutando em 127.0.0.1:65432 ---** e ficará esperando por conexões. **Não feche este terminal; o servidor precisa continuar rodando.**

Iniciar o Cliente: Abra **um segundo terminal** (separado do primeiro). Navegue para o mesmo diretório. Execute o script do cliente:

Bash

```
python cliente_clima.py
```

2. No terminal do cliente, você verá mensagens sobre a coleta de dados do IBGE, a criação das threads e as tentativas de conexão e envio de dados para o servidor. No terminal do servidor, você verá os dados recebidos de cada thread do cliente.

Seguindo esses passos, você poderá observar a interação e a troca de mensagens entre as threads do meu cliente e o servidor TCP. Isso demonstra a comunicação básica e como eu paralelizei as tarefas de coleta e envio de dados.